Golden Bull — Full-Stack Technical Reference

A file-by-file guide to the codebase: what each piece is, why it exists, and what it solves

Contents

Part 0 — Architecture Overview

Golden Bull is a two-project system:

- **Frontend**: Angular (standalone components, signals), deployed as a static build on Vercel.
- **Backend**: Node.js + Express + TypeScript, using Prisma as the ORM against a PostgreSQL database hosted on Neon (a serverless Postgres provider). Also deployed on Vercel, as a serverless function.

They are two *separate* Vercel projects even though they live in one Git repository — the frontend calls the backend over plain HTTPS/JSON, exactly like it would call any third-party API. This matters because:

- They deploy independently. A frontend-only change doesn't require a backend redeploy, and vice versa.
- CORS becomes a real concern (the backend must explicitly allow the frontend's origin — this caused a real production bug covered in Part 6).
- The backend has no idea "who" is calling it beyond what's in the request (headers, JWT token) — there's no shared session state, no cookies by default. This is why JWT (a signed token proving identity) is used instead of server-side sessions.

Why Prisma instead of raw SQL? Prisma is a type-safe query builder/ORM. You describe your data shape once in `schema.prisma`, and Prisma generates a fully-typed TypeScript client (`prisma.product.findMany(...)`, etc.) with autocomplete and compile-time checking of field names — so a typo in a column name is a build error, not a runtime surprise discovered in production.

Why signals in the frontend instead of plain RxJS everywhere? Angular signals are simpler synchronous reactive primitives — you read a signal with `mySignal()`, write it with `mySignal.set(x)`, and Angular's change detection automatically knows when to re-render. RxJS Observables are still used for anything asynchronous (HTTP calls), but the moment that data lands, it's converted into a signal (via `toSignal`) so the rest of the component doesn't need to think about subscriptions/unsubscriptions.

Part 1 — Backend, File by File

1.1 `backend/src/db.ts`

```ts
import { PrismaClient } from '@prisma/client';

// Single shared Prisma instance for the whole process (avoids exhausting
// database connections when routes import this repeatedly).
export const prisma = new PrismaClient();
```

What it does: creates exactly one `PrismaClient` instance and exports it.

Why it exists: every route file needs to talk to the database. If every file created its *own* new `PrismaClient()`, you'd end up with dozens of separate connection pools all fighting for the

same small number of database connections a hobby-tier Postgres allows. By creating it once here and importing `{ prisma }` everywhere else, the whole app shares one client and one connection pool.

Problem it solves: connection exhaustion under load, and (in serverless environments especially) unpredictable behavior from creating new clients on every function invocation.

1.2 backend/src/utils/jwt.ts

```ts
import jwt from 'jsonwebtoken';

export interface JwtPayload {
  sub: string; // user id
  role: string;
}

const SECRET = process.env.JWT_SECRET ?? 'dev-secret-change-me';
const EXPIRES_IN = process.env.JWT_EXPIRES_IN ?? '7d';

export function signToken(payload: JwtPayload): string {
  return jwt.sign(payload, SECRET, { expiresIn: EXPIRES_IN as jwt.SignOptions['expiresIn'] });
}

export function verifyToken(token: string): JwtPayload {
  return jwt.verify(token, SECRET) as JwtPayload;
}
```

What is a JWT? A JSON Web Token is a string with three base64 parts (`header.payload.signature`) that encodes some data (here: user id + role) and is cryptographically signed with a secret key. Anyone can *read* the payload (it's not encrypted, just encoded), but nobody can *forge* a valid one without knowing SECRET, because the signature would not match.

`signToken`: takes `{ sub, role }` (sub = "subject," a JWT convention for "whose token is this") and produces a signed string. This is what gets handed to the frontend after a successful login/register, and the frontend stores it (in `localStorage`, see `auth.service.ts`) and re-sends it on every future request.

`verifyToken`: takes a token string, checks the signature is valid and the token hasn't expired, and returns the decoded payload. If the signature doesn't match or it's expired, `jwt.verify` throws — the caller (`auth.middleware.ts`) catches that and treats it as "not authenticated."

Why a fallback secret (`dev-secret-change-me`)? So the app still boots locally even if you forgot to set JWT_SECRET in `.env` — but in production, JWT_SECRET is set as a real environment variable in Vercel, so the fallback is never actually used there. If it *were* used in production, anyone could forge admin tokens, so this fallback existing is a deliberate "works out of the box locally, but you MUST set a real secret before going live" pattern.

1.3 backend/src/middleware/auth.middleware.ts

What is Express middleware? A function `(req, res, next) => {}` that runs *before* your actual route handler. It can inspect/modify the request, short-circuit with an error response, or call `next()` to let the request continue to the next middleware/route handler in the chain.

```ts
export interface AuthedRequest extends Request {
  user?: { id: string; role: string };
}

export function requireAuth(req: AuthedRequest, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  const token = header?.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) {
    return res.status(401).json({ message: 'Missing or invalid Authorization header' });
  }
  try {
    const payload = verifyToken(token);
    req.user = { id: payload.sub, role: payload.role };
    next();
  } catch {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }
}
```

What it does: looks for an `Authorization: Bearer <token>` header, verifies the token, and if valid, attaches `{ id, role }` onto `req.user` so every route handler downstream can read `req.user!.id` without re-parsing the token itself. If there's no token, or it's invalid/expired, it immediately responds `401 Unauthorized` and never calls `next()` — the route handler never even runs.

```ts
export function optionalAuth(req: AuthedRequest, _res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  const token = header?.startsWith('Bearer ') ? header.slice(7) : null;
  if (token) {
    try {
      const payload = verifyToken(token);
      req.user = { id: payload.sub, role: payload.role };
    } catch {
      // ignore invalid token — treat as guest
    }
  }
  next();
}
```

What it does differently: same idea, but it *always* calls `next()` — whether or not a token was present or valid. If there's a valid token, `req.user` gets set (so a logged-in customer's order links to their account); if there's no token at all, the request just proceeds as an anonymous guest.

Problem it solves: guest checkout. Before this existed, `POST /api/orders` used `requireAuth`, which meant nobody could buy anything without creating an account first. Swapping to `optionalAuth` on that one route made accounts optional while still linking orders to an account *when one exists*.

```ts
export function requireAdmin(req: AuthedRequest, res: Response, next: NextFunction) {
  if (req.user?.role !== 'admin') {
    return res.status(403).json({ message: 'Admin access required' });
  }
  next();
}
```

What it does: checks the role that `requireAuth` already attached onto `req.user` (must run *before* this in the middleware chain — e.g. `productsRouter.post('/', requireAuth, requireAdmin, ...)`, where both run in order). If the user isn't an admin, responds `403 Forbidden`.

Why two separate middlewares (`requireAuth` + `requireAdmin`) instead of one combined function? Composability. `requireAuth` alone is useful for "any logged-in user" routes (e.g. `/orders/me`). Stacking `requireAuth, requireAdmin` for admin-only routes reuses the same building block instead of duplicating the token-parsing logic in a second function.

1.4 backend/src/middleware/error.middleware.ts

```
export class ApiError extends Error {
  constructor(public status: number, message: string) {
    super(message);
  }
}

export function errorHandler(err: unknown, _req: Request, res: Response, _next: NextFuncti
  if (err instanceof ApiError) {
    return res.status(err.status).json({ message: err.message });
  }
  if (err instanceof ZodError) {
    const first = err.issues[0];
    const message = first ? `${first.path.join('.')}: ${first.message}` : 'Invalid request
    return res.status(400).json({ message });
  }
  console.error(err);
  return res.status(500).json({ message: 'Internal server error' });
}

export function notFound(_req: Request, res: Response) {
  res.status(404).json({ message: 'Route not found' });
}
```

`ApiError`: a custom error class carrying an explicit HTTP status code. Anywhere in a route handler, you can `throw new ApiError(404, 'Product not found')` and it will be caught by Express and routed to `errorHandler` below, producing a clean `{ message: "Product not found" }` response with status 404, instead of a generic crash.

`errorHandler`: this is Express's special "error-handling middleware" — Express recognizes it by its four-argument signature `(err, req, res, next)` and routes any error thrown/passed-to-`next()` from anywhere in the app here, instead of to a normal route. It's a cascade of checks: 1. If it's our own `ApiError`, use its exact status/message (these are deliberate, expected error responses we control). 2. If it's a `ZodError` (validation failure — see 1.5 below for what Zod is), extract the *first* validation problem and return it as a `400 Bad Request` with a human-readable message like `"email: Invalid email"`. 3. Otherwise, it's a genuine, unexpected crash — log it to the server console (visible in Vercel's function logs) and return a generic `500` so internal details never leak to the client.

A real bug this fixed: before step 2 existed, a `ZodError` (e.g. someone submitting an invalid email in the contact form) fell all the way through to step 3 and came back as `"Internal server error"` — which looks exactly like a real crash to both the customer and to anyone

debugging it, even though the actual problem was just "this isn't a valid email address." Adding the explicit `ZodError` branch turns confusing 500s into accurate, actionable 400s.

`notFound`: a catch-all for any route that doesn't match anything defined (e.g. a typo'd URL) — registered *after* all the real routes in `index.ts`, so it only runs if nothing else already handled the request.

1.5 What is Zod, and why validate with it?

Every route in this backend defines a schema like:

```
const registerSchema = z.object({
  name: z.string().min(2),
  email: z.string().email(),
  password: z.string().min(6),
});
```

…and then calls `registerSchema.parse(req.body)` at the top of the handler. Zod is a runtime schema/validation library. TypeScript's type system only exists at *compile time* — it cannot stop someone from sending `{ "name": 123 }` over the network at runtime, because by the time the server is running, all the TypeScript type annotations have already been stripped out. Zod re-checks the actual data at runtime and throws a `ZodError` (caught by `error.middleware.ts` above) the instant something doesn't match — wrong type, missing field, invalid email format, etc. — before any of that bad data can reach the database.

1.6 `backend/src/routes/auth.routes.ts`

Registers `/api/auth/*`:

- **`POST /register`**: validates `{ name, email, password }`, checks the email isn't already taken (`prisma.user.findUnique`), hashes the password with `bcrypt.hash(password, 10)` (never store plaintext passwords — bcrypt is a slow, salted one-way hash designed specifically to resist brute-forcing), creates the user, and returns a signed JWT + the public-safe user fields.
- **`POST /login`**: looks up the user by email, uses `bcrypt.compare(password, user.passwordHash)` to check the submitted password against the stored hash (you can never "decrypt" a bcrypt hash — you can only re-hash the guess and compare), and if it matches, signs and returns a token.
- **`GET /me`**: behind `requireAuth` — returns the currently logged-in user's own profile, looked up fresh from the DB each time (not just decoded from the token) so a profile edit is reflected immediately.
- **`PATCH /me`**: updates name/email.
- **`PATCH /me/password`**: requires the *current* password to be re-entered and verified (`bcrypt.compare`) before allowing a new one to be set — this stops someone who has stolen a logged-in browser session from silently taking over the account by changing the password without knowing the old one.

`toPublicUser(user)`: a small helper that whitelists exactly which fields are safe to send to the frontend (`id, name, email, role, createdAt`) — deliberately excluding `passwordHash`. This pattern (an explicit "public shape" function) is safer than just doing `res.json(user)`, which would leak the password hash to anyone who opens their browser's Network tab.

1.7 backend/src/routes/products.routes.ts

Registers `/api/products/*`. Products are the store catalog.

Storage trick — JSON-in-a-column: `colors` and `sizes` are stored as *JSON strings* inside plain `String` Postgres columns (`colors String`), not as separate relational tables. `serialize()` parses them back into real arrays right before sending the response:

```
function serialize(p) {
  return {
    ...p,
    colors: JSON.parse(p.colors),
    sizes: p.sizes ? JSON.parse(p.sizes) : [],
    variants: p.variants ?? [],
  };
}
```

Why not a proper `Color` table? Colors are small, always fetched together with their parent product, and never queried independently ("give me every product that has a Havana color" is not a query this app ever needs) — so the flexibility of a real join is not worth the extra complexity. This is a deliberate, pragmatic tradeoff, not an oversight.

Contrast with `ProductVariant`, which *is* a real related table:

```
model ProductVariant {
  id        Int     @id @default(autoincrement())
  productId Int
  product   Product @relation(fields: [productId], references: [id], onDelete: Cascade)
  color     String
  size      String?
  stock     Int     @default(0)
}
```

This one needed to be a real table because each row needs its *own* mutable number (`stock`) that changes independently of the others — you can't cleanly do "increment the stock of just the Black/42 combination" if it's buried inside one big JSON blob you'd have to parse, mutate, and rewrite as a whole. `onDelete: Cascade` means deleting a product automatically deletes all of its variant rows too, so you never end up with orphaned variant rows pointing at a product that no longer exists.

The `PUT /:id` transaction:

```
const updated = await prisma.$transaction(async (tx) => {
  await tx.product.update({ where: { id }, data });
  if (variants) {
    await tx.productVariant.deleteMany({ where: { productId: id } });
    await tx.productVariant.createMany({ data: variants.map(...) });
  }
  return tx.product.findUnique({ where: { id }, include: { variants: true } });
});
```

What is a transaction, and why is one needed here? A transaction bundles several database operations so they either *all* succeed or *all* roll back together — there's no in-between state visible to anyone else. Here: update the product's own fields, wipe out its old variant rows, and insert the new set. If step 2 succeeded but step 3 crashed (e.g. bad data), without a transaction you'd be left with a product that has *zero* variants (all stock apparently gone) until someone fixes it manually. Wrapping all three in `$transaction` guarantees that never happens — either the whole update lands, or none of it does.

Why delete-then-recreate variants instead of updating them individually? The admin form re-sends the *entire* color×size stock grid on every save, with no stable IDs from the frontend's side (the grid is rebuilt from the `colors`/`sizes` inputs every time you type). Rather than trying to diff "what changed" against the old rows, it's simpler and just as correct to replace the whole set atomically.

1.8 backend/src/routes/orders.routes.ts

Registers `/api/orders/*`. This is the most business-logic-heavy route file.

`resolveDiscount(code, subtotal)`:

```
async function resolveDiscount(code, subtotal) {
  if (!code) return { discountCode: null, discountAmount: 0 };
  const found = await prisma.discountCode.findUnique({ where: { code: code.toUpperCase() }
  if (!found || !found.active) throw new ApiError(400, 'Invalid or expired discount code')
  if (found.expiresAt && found.expiresAt < new Date()) throw new ApiError(400, 'This disco
  if (found.maxUses != null && found.usedCount >= found.maxUses) {
    throw new ApiError(400, 'This discount code has reached its usage limit');
  }
  const discountAmount = Math.round(subtotal * (found.percent / 100));
  return { discountCode: found.code, discountAmount, discountId: found.id };
}
```

This is deliberately re-run *again* on the server at the moment an order is actually placed, even though the frontend already called `/discounts/validate` earlier to show a live preview. **Never trust client-side validation for anything that affects money** — a customer could tamper with the request in their browser's dev tools and claim a 90% discount that was never actually validated. Re-checking server-side, right before the charge, is the only trustworthy point.

`POST /` (create order) — uses `optionalAuth` (guest checkout, explained in 1.3), computes `subtotal` by re-summing `price * quantity` for every item **from the request body itself** (not from a trusted price the frontend "promises" — same reasoning as above: never trust client-supplied totals), resolves any discount, computes the 20% deposit, creates the `Order` + nested `OrderItem` rows in one `prisma.order.create(...)` call (Prisma lets you create a parent and its children in a single nested write), and finally increments the discount code's `usedCount` if one was used — enforcing `maxUses` limits over time.

`GET /me`: a logged-in customer's own order history (`requireAuth` — a guest has no account to look anything up under).

`GET /` (admin): every order in the system, `requireAuth + requireAdmin`, including the linked user's public info via Prisma's `include: { user: { select: {...} } }` — a *selective* include, so it only pulls `id/name/email` off the related user row, not the password hash.

`PATCH /:id/status`: admin-only, validates the new status against a fixed Zod enum (`pending | confirmed | shipped | outForDelivery | delivered | cancelled`) so a typo or arbitrary string can never be written into the database as a "status" — the customer-facing stepper component (Part 4) depends on the status always being one of these exact known values.

`GET /:id`: fetches a single order, but then explicitly checks `order.userId !== req.user!.id && req.user!.role !== 'admin'` and throws 403 otherwise — this stops customer A from guessing customer B's order ID and viewing their address/phone number. This check is called

an **authorization check** (are you *allowed* to see this specific resource), distinct from **authentication** (are you logged in at all, checked earlier by `requireAuth`) — a very common source of real security bugs is doing authentication but forgetting this second, resource-specific check.

1.9 backend/src/routes/discounts.routes.ts

Registers `/api/discounts/*`.

`generatePromoCode()`: builds a random 6-character code from a deliberately restricted alphabet that excludes `0/O/1/I` — characters that are easy to visually confuse when a customer is copying a code off an email onto a checkout screen.

`POST /subscribe`: the "Subscribe & get 10% off" flow. Normalizes the email (`trim().toLowerCase()`, so `Test@X.com` and `test@x.com` are treated as the same person), checks if this email already has a code (`ownerEmail` is a `@unique` column, so this is a fast indexed lookup) — if so, it's **idempotent**: re-submitting the same email just returns their existing code instead of creating a second one or erroring. Otherwise it generates a code (retrying up to 5 times against extremely unlikely collisions), creates a `DiscountCode` row tied to that email, sends the code by email (`sendEmail`, see 1.11), and separately notifies the admin (`notifyAdmin`) that a new subscriber signed up.

`POST /validate`: the "preview" endpoint the checkout page's Apply button calls, so the customer sees the discounted total *before* placing the order. Deliberately duplicates the same checks as `resolveDiscount` in `orders.routes.ts` — that duplication is intentional, not an oversight: this endpoint's whole purpose is to preview without side effects, while the one in `orders.routes.ts` is the one whose result is trusted and actually applied.

Admin CRUD (`GET /`, `POST /`, `PATCH /:id`): list/create/toggle discount codes. Note there's no dedicated admin *page* for this in the frontend yet — these endpoints exist and work, but must currently be called directly (e.g. with a REST client) rather than through a UI form.

1.10 backend/src/routes/contact.routes.ts

Registers `/api/contact/*` — the public "Contact Us" form on the homepage.

```
const contactSchema = z.object({
  firstName: z.string().min(1),
  lastName: z.string().min(1),
  phone: z.string().min(6),
  email: z.string().email().optional().or(z.literal('')),
  message: z.string().min(1),
});
```

`z.string().email().optional().or(z.literal(''))`: reads as "either a valid email, or an empty string" — this is how the email field stays *optional* (an empty string from an untouched form field is fine) while still rejecting genuinely malformed input like a garbage non-email string the instant it's non-empty. This exact validator is what turns a bad-input mistake into a clean `400` (see 1.4's `ZodError` handling) instead of a `500`.

`POST /`: public — anyone can submit. Creates a `ContactMessage` row and calls `notifyAdmin(...)` so a real message shows up in the admin's inbox almost immediately.

`GET /`, `PATCH /:id/read`, `DELETE /:id`: all `requireAuth + requireAdmin` — only the admin dashboard's Messages page can list, mark-as-read, or delete these.

1.11 backend/src/utils/email.ts

```
const RESEND_API_KEY = process.env.RESEND_API_KEY;
const FROM = process.env.RESEND_FROM || 'Golden Bull <onboarding@resend.dev>';
const ADMIN_EMAIL = process.env.ADMIN_NOTIFY_EMAIL || 'admin@goldenbull.com';

export async function sendEmail(to: string, subject: string, html: string): Promise<void>
  if (!RESEND_API_KEY) {
    console.log(`[email:not-sent, no RESEND_API_KEY set] to=${to} subject="${subject}"`);
    return;
  }
  const res = await fetch('https://api.resend.com/emails', {
    method: 'POST',
    headers: { Authorization: `Bearer ${RESEND_API_KEY}`, 'Content-Type': 'application/jso
    body: JSON.stringify({ from: FROM, to, subject, html }),
  });
  if (!res.ok) console.error('Resend email failed:', res.status, await res.text());
}

export async function notifyAdmin(subject: string, html: string): Promise<void> {
  await sendEmail(ADMIN_EMAIL, subject, html);
}
```

What it does: a thin wrapper around Resend's plain HTTP email API — deliberately using the built-in `fetch` instead of installing Resend's official SDK package, since the entire integration is one HTTP POST and doesn't need a whole extra dependency.

The "fail open" design: if `RESEND_API_KEY` isn't configured, `sendEmail` doesn't throw — it just logs a note and returns successfully. This means the rest of the app (e.g. the newsletter-signup endpoint) keeps working — the discount code still gets generated and saved — even before Resend has been set up. The email is a "nice to have" side effect, not a hard dependency the core feature should crash without.

`notifyAdmin`: a convenience wrapper that always sends to the one configured admin address, so every "the admin should know about this" call site (new contact message, new subscriber) doesn't need to repeat the same `ADMIN_NOTIFY_EMAIL` env var lookup.

1.12 backend/src/index.ts

This is the actual server entry point — where the Express app is assembled and started.

```
const allowedOrigins = (process.env.CORS_ORIGIN ?? 'http://localhost:4200')
  .split(',')
  .map((o) => o.trim());

app.use(
  cors({
    origin(origin, callback) {
      const isLocalhost = !origin || /^https?:\/\/(localhost|127\.0\.0\.1)(:\d+)?$/.test(o
      const isVercelPreview = origin ? /^https:\/\/[a-z0-9-]+\.vercel\.app$/.test(origin)
      if (isLocalhost || isVercelPreview || allowedOrigins.includes(origin ?? '')) {
```

```
        callback(null, true);
      } else {
        callback(new Error(`CORS blocked for origin: ${origin}`));
      }
    },
  })
);
```

What is CORS, concretely? By default, a browser refuses to let JavaScript running on the frontend's domain read the response from a fetch to the backend's *different* domain **unless** that second server explicitly says, via an `Access-Control-Allow-Origin` response header, "yes, requests from that origin are allowed." The `cors` npm package here generates that header automatically, but only for origins the `origin()` callback approves.

Why the regex for `*.vercel.app`? Every Vercel *preview* deployment (one per git branch/PR) gets its own random subdomain. There's no fixed list of these to add to `CORS_ORIGIN` one at a time as new branches come and go — so the check is loosened to "any subdomain of vercel.app" instead, which is safe here specifically because all of those subdomains are deployments of *this same account's own projects*, not arbitrary third parties.

A real production bug this explains: before this regex existed, only one exact production URL was in `allowedOrigins`. Testing from any preview-branch URL got silently blocked by the browser before the request even reached the Express error handler — which is why the browser's dev tools showed *both* "CORS error" *and* a confusing 500, since the crash (`callback(new Error(...))`) itself doesn't carry CORS headers back, compounding the failure.

```
app.use(express.json({ limit: '6mb' }));
```

Raised from Express's small default body-size limit (100kb) specifically because checkout can attach a base64-encoded payment-proof screenshot in the JSON body — a compressed photo easily exceeds 100kb.

```
app.use('/api/auth', authRouter);
app.use('/api/products', productsRouter);
app.use('/api/orders', ordersRouter);
app.use('/api/discounts', discountsRouter);
app.use('/api/contact', contactRouter);

app.use(notFound);
app.use(errorHandler);
```

Order matters here. Express matches middleware/routes top-to-bottom. Mounting `notFound` after all the real routers means it only ever runs if nothing above it matched. Mounting `errorHandler` dead last means it can catch errors thrown anywhere above it — an error-handling middleware (recognizable by its 4 arguments) must be registered after everything it's meant to protect.

1.13 backend/prisma/schema.prisma

This file is the single source of truth for the database shape. Prisma reads it and: 1. Generates the fully-typed `PrismaClient` (`npx prisma generate`). 2. Can push that shape directly onto the real database (`npx prisma db push`) — this is what actually creates/alters tables and columns.

Key models and the *reasoning* behind field choices (not just what they are):

- **User**: `role String @default("customer")` is a plain string, not a real Postgres enum — kept intentionally simple since there are only ever two values (`"customer"` / `"admin"`) checked by string comparison in `requireAdmin`.
- **Product**: see 1.7 for the JSON-column reasoning on `colors`/`sizes`.
- **Order**: `userId String?` (nullable) is what makes guest checkout possible at the database level — without the `?`, Postgres would reject any order row that didn't have a real user id, making guest orders impossible no matter what the API code did. `shippingGovernorate` etc. are flat strings (not a foreign key to some `Address` table) because an order's shipping address is a point-in-time snapshot — if the customer later edits their saved address, past orders must still show the address that was actually used *at the time*, not "whatever their address is now."
- **DiscountCode**: `ownerEmail String? @unique` — nullable *and* unique. Nullable because the shared `WELCOME10` code has no single "owner" (`ownerEmail: null`), but a code auto-generated from a newsletter signup does. `@unique` is what makes the "does this email already have a code?" lookup in `discounts.routes.ts` both correct (never two codes per email) and fast (Postgres builds an index for unique columns automatically).
- **ProductVariant**: see 1.7.
- **ContactMessage**: `read Boolean @default(false)` — every new message defaults to unread, which is what drives the notification badge in the admin dashboard bell icon.

1.14 backend/prisma/seed.ts

This script populates (or re-populates) the database with the real product catalog, a test customer, an admin account, and the `WELCOME10` discount code. Run manually with `npm run seed`.

A real, serious bug that lived here and was fixed: the original version started with:

```
await prisma.orderItem.deleteMany();
await prisma.order.deleteMany();
await prisma.product.deleteMany();
await prisma.user.deleteMany();
```

This *unconditionally wiped every order, every user, and every product* every single time the script ran — which was fine the very first time (empty database), but became actively destructive the moment real customer orders existed: re-running the seed (e.g. after adding a new field to `Product`) silently deleted real customers' order history. This is exactly the kind of bug that doesn't show up in testing (an empty dev database never has orders to lose) and only bites in production.

The fix: replaced the delete-everything approach with **upserts** keyed on a natural, stable identifier instead of the auto-incrementing `id`:

```
const existing = await prisma.product.findFirst({ where: { nameEn: p.nameEn } });
if (existing) {
  await prisma.product.update({ where: { id: existing.id }, data });
} else {
  await prisma.product.create({ data });
}
```

For users: `prisma.user.upsert({ where: { email }, update: {}, create: {...} })` — Prisma's built-in `upsert` does the same "find or create" in one call when there's a genuinely unique column (`email`) to key off of. Products needed the manual find-then-update/create

version instead, because `nameEn` isn't declared `@unique` in the schema (so `upsert`'s `where` clause can't use it directly).

Why this matters as a general lesson: any script that runs more than once against a database holding real, relationship-linked data (an `Order` has a foreign key to `Product`) must either (a) never delete rows other things depend on, or (b) explicitly cascade the deletion in a way you actually intend. "Reset everything and start over" is a fine strategy for a brand-new project with no real users yet — it stops being fine the moment it isn't.

Part 2 — Frontend Core Concepts

2.1 Standalone components

Every component in this project (e.g. `@Component({ selector: 'app-navbar', imports: [...], ... })`) is a **standalone component** — Angular's newer style that doesn't require registering components inside an `NgModule`. Each component directly lists exactly which other components/directives/pipes it needs in its own `imports: []` array. This makes dependencies explicit and local — you can look at any single component file and know exactly what it needs, instead of hunting through a separate module file.

2.2 Signals

```
count = signal(0);
doubled = computed(() => this.count() * 2);

increment() { this.count.update(n => n + 1); }
```

- `signal(initialValue)` creates a reactive container. Read it by *calling* it as a function: `this.count()`.
- `.set(x)` replaces the value outright; `.update(fn)` replaces it based on the previous value.
- `computed(() => ...)` derives a new signal from others — it automatically re-runs whenever any signal it reads from changes, and Angular's templates automatically re-render wherever that computed signal is used, with no manual subscription/unsubscription bookkeeping required.

This project uses signals for essentially all local UI state — cart contents (`cart.service.ts`), loading flags, form-adjacent state like "is this promo code currently being checked."

2.3 RxJS + Signals interop

HTTP calls in Angular (`HttpClient.get(...)`) return RxJS `Observables`, not signals — because a network request is asynchronous and can be cancelled/retried/combined in ways signals alone don't model. The bridge between the two worlds:

```
filteredProducts = toSignal(
  toObservable(this.activeFilter).pipe(
    switchMap((filter) => this.productsService.getByCategory(filter))
  ),
  { initialValue: [] }
);
```

- `toObservable(someSignal)` turns a signal *back into* an Observable stream of its changing values — here, every time `activeFilter` changes (user clicks a different category tab), a new value flows through.
- `switchMap` is the key RxJS operator: it says "when a new filter comes in, cancel whatever previous HTTP request was still in flight, and start a new one." Without this, rapidly clicking between category tabs could let an old, slow request's response arrive *after* a newer one and incorrectly overwrite the list with stale data — `switchMap` guarantees only the most recent request's result ever wins.
- `toSignal(observable, { initialValue })` converts the whole pipeline back into a signal the template can read synchronously (`filteredProducts()`), with `initialValue` covering the moment before the very first HTTP response has arrived.

2.4 HTTP Interceptors

An interceptor is a function that every single `HttpClient` request passes through, regardless of which service/component made it — used here for two cross-cutting concerns that would be repetitive to handle individually in every service:

- **`auth.interceptor.ts`**: attaches `Authorization: Bearer <token>` automatically to every request going to our own API, so no individual service method has to remember to do this itself.
- **`loading.interceptor.ts`**: increments/decrements a shared counter around every request's lifetime, driving the global top progress bar (Part 4) without any component needing to manually show/hide it.

2.5 Guards

A route guard (`CanActivateFn`) is a function Angular's router calls *before* navigating to a route — it can allow the navigation, or redirect elsewhere instead.

```
export const authGuard: CanActivateFn = (_route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isLoggedIn()) return true;
  return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
};
```

Three guards exist: `authGuard` (must be logged in — used on `/profile`), `guestGuard` (must NOT be logged in — used on `/login`/`/register`, so an already-logged-in user gets bounced to their profile instead of seeing a login form again), and `adminGuard` (must have `role === 'admin'` — used on every `/admin/**` route).

2.6 Transloco (i18n)

Every user-facing string goes through a translation function, e.g. `{{ t('checkout.placeOrder') }}` in templates, backed by two flat JSON dictionaries (`public/i18n/en.json`, `public/i18n/ar.json`). Switching languages instantly re-renders every `t(...)` call across the whole app with no page reload, and also flips the document's `dir` attribute between `ltr`/`rtl` (handled centrally in `app.ts`) so layout mirrors correctly for Arabic.

Part 3 — Frontend Core Services, File by File

3.1 core/services/auth.service.ts

Holds the single source of truth for "who is currently logged in," backed by `localStorage` so it survives page refreshes.

```
private _user = signal<User | null>(this.loadUser());
readonly user = this._user.asReadonly();
readonly isLoggedIn = computed(() => this._user() !== null);
readonly isAdmin = computed(() => this._user()?.role === 'admin');
```

`asReadonly()` is deliberate: components can *read* `auth.user()` freely, but can't call `.set()` on it directly from outside the service — the only way to change it is through the service's own methods (`login`, `logout`, etc.), keeping the "how does the current user change" logic in one place.

`login`/`register` call the backend, then persist both the token and user object into `localStorage` and update the signal — this is the one function where the frontend's notion of "logged in" and the backend's JWT become the same thing.

3.2 core/services/cart.service.ts

The shopping cart, persisted to `localStorage` so it survives a refresh (an `effect` re-runs the storage write whenever the cart signal changes).

`variantKey`: `${id}-${color}-${size}` — the cart's real "identity" for a line item isn't just the product id, it's the *combination* of product + color + size, because a customer can legitimately have both a Black/42 and a Brown/41 pair of the same slipper in their cart as two separate lines. Using the plain product id as the key would silently merge those into one line and lose the color/size distinction entirely.

3.3 core/services/products.service.ts / orders.service.ts / contact.service.ts / discounts.service.ts

These four are thin, symmetrical wrappers around `HttpClient`, one per backend resource — each just exposes typed methods (`getAll()`, `create()`, …) that call the matching REST endpoint documented in Part 1. Keeping them this thin (no business logic, just typed HTTP calls) means components never construct URLs or deal with raw `HttpClient` directly — if a backend URL path ever changes, there's exactly one place to update it.

3.4 core/services/admin-notifications.service.ts

Polls `orders.getAll()` and `contact.getAll()` every 30 seconds while an admin is logged in, and exposes computed counts (`pendingOrdersCount`, `unreadMessagesCount`, `totalBadge`) that drive the bell icon badge in the admin shell. `start()` guards against being called more than once since the admin shell component could otherwise re-trigger it on every navigation within `/admin/**`.

3.5 core/services/loading.service.ts

```
readonly active = signal(false);
readonly visible = signal(false);
```

```
start() {
  this._requestCount.update(n => n + 1);
  this.active.set(true);
  if (!this._showTimer) {
    this._showTimer = setTimeout(() => { if (this.active()) this.visible.set(true); }, 200
  }
}
```

Two separate signals on purpose. `active` flips true the instant *any* request starts. `visible` only flips true if a request is *still* in flight 200ms later. **Why the delay?** Most requests resolve in well under 200ms — showing a progress bar/blur overlay for a blink-and-you-miss-it instant reads as flickering, not as helpful feedback. Debouncing the *visible* signal (while still tracking the true state instantly in `active`) means fast requests show nothing at all, and only genuinely slow ones show the loading UI — this is what the global top progress bar (Part 4) actually binds to.

3.6 `core/services/theme.service.ts` / `announcement.service.ts`

Small, single-purpose signal stores: `theme.service.ts` toggles a `data-theme` attribute between `"dark"`/`"light"` (the actual color values live in `styles.scss` as CSS custom properties keyed off that attribute); `announcement.service.ts` tracks whether the site-wide announcement bar has been dismissed for this browser session (`sessionStorage`, not `localStorage` — deliberately reappearing on a brand-new session/tab rather than staying dismissed forever).

3.7 `core/guards/auth.guard.ts`, `core/interceptors/*`, `core/utils/http-error.ts`

Covered conceptually in 2.4/2.5. `http-error.ts` is a small helper (`getErrorMessage(err, fallback)`) that digs a human-readable message out of an Angular `HttpErrorResponse` (preferring the backend's own `{ message }` JSON body, since that's exactly what `error.middleware.ts` on the backend always sends), falling back to a generic message if the response isn't shaped as expected (e.g. the request never reached the server at all).

3.8 `core/constants/governorates.ts` / `legal-content.ts`

Plain data files, not services — `governorates.ts` is the list of Egyptian governorates with an estimated courier fee each (shown at checkout and on the shipping-policy page, recalculated for a Tanta/Gharbia shipping origin rather than Cairo), and `legal-content.ts` holds the full bilingual text for the four legal pages (shipping/privacy/terms/reviews), kept out of the i18n JSON files specifically because that long-form legal text would otherwise bloat every other translation lookup in the app.

Part 4 — Shared Components

These live under `@shared/` because more than one feature page uses them (as opposed to `@features/`, where each folder is a single page/route).

- **`navbar`**: the storefront's top navigation. Renders a *different* set of links depending on `auth.isAdmin()` — this is where an earlier "navbar shows duplicated links" bug lived, fixed by branching the whole nav on admin-vs-customer instead of just appending admin links onto the customer set.

- **footer**: site footer, including the newsletter/promo-code signup form wired to `discounts.service.ts`.
- **announcement-bar**: the dismissible gold ticker bar advertising the promo code, driven by `announcement.service.ts`.
- **cart-drawer**: the slide-out sidebar cart (as opposed to the full `/cart` page) — opened via `cart.toggleDrawer()`, called from the navbar's cart icon.
- **top-progress-bar**: the global loading indicator described in 3.5 — a slim animated gold bar plus a very light `backdrop-filter: blur()` overlay, both bound to `loading.visible()`, with `pointer-events: none` so it never blocks clicks even while visible.
- **order-status-stepper**: a visual progress tracker mapping the backend's raw `status` string onto four customer-facing stages (`pending`/`confirmed` both collapse into one "working on your piece" stage, then `shipped` → `outForDelivery` → `delivered`), with a distinct red "Cancelled" state instead of a stepper when `status === 'cancelled'`.
- **admin-shell**: the sidebar+topbar layout wrapping every `/admin/**` page, including the notification bell dropdown fed by `admin-notifications.service.ts`. Its dropdown background was changed from `var(--bg-card)` (a semi-transparent tint meant for backgrounds, which looked washed-out as a floating popup) to `var(--bg-secondary)` (a solid color), and item text from `var(--text-secondary)` to `var(--text-primary)` for readability.

Part 5 — Feature Pages

- **home**: the landing page — hero carousel, value props, categories, featured products, testimonials, and the Contact Us form (wired to `contact.service.ts`).
- **products**: the catalog grid with category tabs. Each product card's color swatches are independently clickable, swapping that card's own thumbnail and carrying the chosen color into the product-detail page via a query param — fixing an earlier bug where the swatches were purely decorative and a product always opened on its default color regardless of which swatch was visually shown.
- **product-detail**: single product page — color/size pickers, quantity stepper, and a loading-vs-not-found distinction (a dedicated `loading` signal) that prevents the page from briefly rendering "Product not found" while the real request is still in flight.
- **cart**: the full cart page (as opposed to the drawer) — reachable via "View full cart," mainly used right before checkout to review line items in detail.
- **checkout**: the most complex page — structured address form, payment method selection, payment-proof upload, promo code application, shipping-fee estimate (informational, not charged online), and the post-order success screen with the status stepper.
- **auth/login, auth/register, forgot-password**: standard credential forms calling `auth.service.ts`.
- **profile**: tabbed account page — profile info, order history (with the status stepper per order), and settings (change password). The "My Orders" tab is hidden entirely for admin accounts, since an admin manages the store rather than shopping in it (see `@if (!auth.isAdmin())` in `profile.html`).
- **admin-orders**: table of every order with an inline status `<select>`, payment-proof lightbox viewer.
- **admin-products**: full CRUD for the catalog, including the per-color/size stock grid described in Part 1.7.
- **admin-messages**: contact-form inbox — mark-as-read on click, delete.
- **admin-dashboard**: the admin landing page — summary stat cards (pending orders, unread messages, product count, revenue) plus "recent orders"/"recent messages" quick-

glance panels.
- **legal-page**: one shared component rendering all four legal pages (shipping/privacy/terms/reviews), picking which bilingual content block to show based on route data.

Part 6 — End-to-End Walkthroughs

6.1 A guest places an order

1. **Frontend** — `checkout.ts` builds a `CreateOrderPayload` from the reactive form + cart signal, and calls `orders.create(payload)`.
2. That HTTP request passes through **`auth.interceptor.ts`** — since there's no logged-in user, there's no token to attach, so the request goes out with no `Authorization` header at all.
3. It also passes through **`loading.interceptor.ts`**, incrementing the shared counter — if this request takes over 200ms, the global progress bar appears.
4. **Backend** — `POST /api/orders` is guarded by `optionalAuth`, not `requireAuth` — no token means `req.user` stays undefined, and the request proceeds anyway.
5. The handler re-computes the subtotal from the request's own line items (never trusting a client-supplied total), calls `resolveDiscount()` if a promo code was included, computes the 20% deposit, and creates the `Order` + `OrderItem` rows — with `userId` evaluating to null for this guest.
6. The response comes back `201 Created` with the full order object.
7. **Frontend** — `checkout.ts`'s subscribe handler marks the order as placed and clears the cart; the template swaps to the success screen, showing the order-status stepper at its very first stage, "working on your piece."

6.2 Admin advances an order's status

1. Admin opens `/admin/orders` (routed through `adminGuard`, which checks `auth.isAdmin()` before the route even loads).
2. Changes the status `<select>` for one order to `"shipped"` — calls `orders.updateStatus(id, 'shipped')`.
3. **Backend** — `PATCH /:id/status` runs `requireAuth` then `requireAdmin` in sequence; a non-admin token would be rejected by the second middleware before the handler ever runs. Zod validates `"shipped"` is one of the known enum values, then updates the row.
4. Back on the frontend, the next time the affected customer opens their `/profile` "My Orders" tab (or the checkout success screen if still on it), the order-status stepper re-renders with the dot for "Handed to shipping company" now filled in gold — purely a function of the raw `status` string now being `"shipped"`.

6.3 Newsletter signup → real promo code → email

1. Customer types an email into the footer's newsletter form; `footer.ts`'s subscribe method calls `discounts.service.ts`'s `subscribe(email)`.
2. **Backend** — `POST /api/discounts/subscribe` normalizes the email, checks for an existing code tied to it (idempotent — resubmitting is safe), generates a fresh 6-character code otherwise, creates the `DiscountCode` row, and calls both `sendEmail()` (to the customer, with their code) and `notifyAdmin()` (to the shop owner, noting a new subscriber).
3. If `RESEND_API_KEY` isn't configured yet, `sendEmail` silently no-ops (logs a note server-side) rather than failing the whole request — the code still gets created and returned to the frontend either way.

4. **Frontend** — the footer shows the returned code inline regardless of whether the email actually sent, since the code is real and usable either way.
5. Later, at checkout, typing that code into the promo field calls `POST /api/discounts/validate`, which finds it by its unique `code` column, checks `active`/`expiresAt`/`maxUses`, and returns the computed discount amount for a live preview — with the *real*, order-affecting check happening again inside `orders.routes.ts` at the moment the order is actually placed (6.1).

End of reference. This document intentionally favors "why" over "what" wherever the two diverge — the code itself is the ground truth for exact syntax; this book exists to explain the reasoning that isn't visible just from reading the code cold.